Fiche de révision : rôles, Gates et Policies (JobPilot)

> **Comment lire cette fiche :** chaque partie tient en quelques lignes. Lis d'abord **« L'essentiel en 30 secondes »**, puis va directement à la partie qui t'intéresse grâce au sommaire.

Sommaire

0. L'essentiel en 30 secondes
1. Les mots à connaître
2. Le rôle
3. Le middleware
4. La Gate
5. La Policy
6. Où vérifier un droit ?
7. Exemple complet : modifier une offre
8. Questions probables à l'évaluation
9. Mémo final (à relire juste avant)

0. L'essentiel en 30 secondes

- **Authentification** = *qui es-tu ?* → middleware `auth` → sinon **redirection vers /login**
- **Autorisation** = *as-tu le droit ?* → middleware de rôle / Gate / Policy → sinon **erreur 403**
- **Rôle** = le *type* d'utilisateur (`admin`, `company`, `candidat`), stocké dans `users.role`
- **Middleware** = barrière **devant une route**, vérifie le **rôle**
- **Gate** = une **règle nommée** (une fonction qui renvoie `true` ou `false`)
- **Policy** = une **classe de règles pour un modèle** (ex. : `OfferPolicy` pour `Offer`)

Phrase clé :

> Le **rôle** dit « tu es une entreprise ». La **Policy** dit « c'est **TON** offre ».

1. Les mots à connaître

Mot	Définition simple	Dans JobPilot
Authentification	Vérifier **qui** est l'utilisateur	Fortify (login) + middleware `auth`
Autorisation	Vérifier **ce qu'il a le droit** de faire	`IsAdmin`, `IsCompany`, `IsCandidat`
Rôle	Catégorie d'utilisateur	colonne `users.role`
Middleware	Code exécuté **avant** le contrôleur	`app/Http/Middleware/`
Gate	Règle d'autorisation **avec un nom**	aucune pour l'instant
Policy	Règles d'autorisation **d'un modèle**	`app/Policies/` (pas encore utilisées)
403	« Interdit » : connecté mais pas le droit	`abort(403)`
401 / redirection login	Pas connecté	middleware `auth`

2. Le rôle

C'est quoi ? Une colonne texte dans la table `users`.

```
// database/migrations/0001_01_01_000000_create_users_table.php
$table->enum('role', ['candidat', 'company', 'admin'])->default('candidat');
```

Comment il est choisi ? À l'inscription, dans `app/Actions/Fortify/CreateNewUser.php` :

```
'role' => ['required', Rule::in(['candidat', 'company'])],  // "admin" interdit
```

➡️ On ne peut pas s'inscrire `admin` : l'admin est mis **à la main en base**.

Comment on le lit ?

```
auth()->user()->role   // 'admin' ou 'company' ou 'candidat'
```

À retenir : le rôle est **relu en base à chaque requête** (session Laravel, pas de token).

3. Le middleware

C'est quoi ? Une barrière placée **devant une route**. Elle s'exécute **avant le contrôleur**.

Le code (JobPilot) : `app/Http/Middleware/IsCompany.php`

```
public function handle(Request $request, Closure $next): Response
{
    if (!auth()->check() || auth()->user()->role !== 'company') {
        abort(403, 'Accès interdit');   // bloqué
    }
    return $next($request);             // autorisé : on continue
}
```

Étape 1 : on lui donne un nom (alias) dans `bootstrap/app.php`

```php
        $middleware->alias([
            'IsAdmin'     => IsAdmin::class,
            'IsCompany'   => IsCompany::class,
            'IsCandidat' => IsCandidat::class,
        ]);
```

Étape 2 : on l'utilise sur des routes dans `routes/web.php`

```php
        Route::middleware(['auth', 'IsCompany'])->group(function () {
            Route::resource('offer', OfferController::class)
                ->only(['create', 'store', 'edit', 'update']);
        });
```

Ordre d'exécution : `auth` d'abord (pas connecté → login), puis `IsCompany` (pas entreprise → 403).
⚠️ **Limite :** le middleware connaît **l'utilisateur**, mais **pas la donnée**. Une entreprise A peut donc modifier l'offre de l'entreprise B. ➡️ Pour corriger ça, on utilise une **Policy**.

4. La Gate

C'est quoi ? Une **règle avec un nom** : une fonction qui renvoie `true` (autorisé) ou `false` (refusé).
Où on la déclare ? Dans `app/Providers/AppServiceProvider.php`, méthode `boot()` :

```php
        use Illuminate\Support\Facades\Gate;

        public function boot(): void
        {
            Gate::define('access-admin', function (User $user) {
                return $user->role === 'admin';
            });
        }
```

➡️ Laravel passe **automatiquement** l'utilisateur connecté dans `$user`.

Comment on l'utilise ?

```php
        Gate::allows('access-admin')      // true ou false
        Gate::denies('access-admin')      // l'inverse
        Gate::authorize('access-admin');  // erreur 403 si refusé
```

```blade
@can('access-admin')
    <a href="{{ route('company.index') }}">Entreprises</a>
@endcan
```

Quand l'utiliser ? Pour une action **qui n'est pas liée à un modèle** (ex. : accéder à l'espace admin).

5. La Policy

C'est quoi ? Une **classe** qui regroupe **toutes les règles d'un modèle**. Une méthode = une action.

Créer une Policy :

```
php artisan make:policy OfferPolicy --model=Offer
```

Comment Laravel la trouve ? Grâce au **nom** (rien à configurer) :

```
App\Models\Offer  ⟶  App\Policies\OfferPolicy
```

Les méthodes et les actions qu'elles contrôlent :

Méthode	Action
`viewAny`	voir la liste (`index`)
`view`	voir une offre (`show`)
`create`	créer (`create` , `store`)
`update`	modifier (`edit` , `update`)
`delete`	supprimer (`destroy`)

Exemple pour JobPilot (`app/Policies/OfferPolicy.php`) :

```php
class OfferPolicy
{
    // Exécutée AVANT toutes les autres : l'admin a tous les droits
    public function before(User $user, string $ability): ?bool
    {
        if ($user->role === 'admin') {
            return true;
        }
        return null;   // null = on laisse la méthode normale décider
    }

    // Seules les entreprises créent des offres
    public function create(User $user): bool
    {
        return $user->role === 'company';
    }

    // Une entreprise ne modifie QUE ses offres
    public function update(User $user, Offer $offer): bool
    {
        return $user->role === 'company'
            && $user->company_id === $offer->company_id;
    }
}
```

Quand l'utiliser ? Dès que la règle dépend **d'une donnée précise** (« cette offre est-elle à moi ? »).

6. Où vérifier un droit ?

Endroit	Code	Protège vraiment ?
Route	`->middleware('IsCompany')` ou `->can('update', 'offer')`	✅ Oui
Contrôleur	`Gate::authorize('update', $offer);`	✅ Oui
FormRequest	`return $this->user()->can('update', $this->route('offer'));`	✅ Oui
Vue Blade	`@can('update', $offer) ... @endcan`	❌ Non : ça **cache seulement** le bouton

⚠️ **Piège classique :** cacher un bouton avec `@can` **ne protège pas** la route. Quelqu'un peut taper l'URL à la main. Il faut **toujours** vérifier aussi côté serveur.

Action sans objet (create) : on passe la **classe**, pas un objet.

```
        Gate::authorize('create', Offer::class);
```

7. Exemple complet : modifier une offre

Objectif : une entreprise ne peut modifier **que ses propres offres**.

① **Route** (`routes/web.php`) : il faut être connecté et être une entreprise

```
        Route::middleware(['auth', 'IsCompany'])->group(function () {
            Route::resource('offer', OfferController::class)->only(['edit', 'update']);
        });
```

② **Policy** (`app/Policies/OfferPolicy.php`) : l'offre doit être à moi

```
        public function update(User $user, Offer $offer): bool
        {
            return $user->company_id === $offer->company_id;
        }
```

③ **Contrôleur** (`app/Http/Controllers/OfferController.php`) : on appelle la Policy

```php
        use Illuminate\Support\Facades\Gate;

        public function edit(Offer $offer)
        {
            Gate::authorize('update', $offer);   // 403 si ce n'est pas mon offre
            return view('offer.edit', compact('offer'));
        }

        public function update(Request $request, Offer $offer)
        {
            Gate::authorize('update', $offer);
            $offer->update($request->validate([
                'Name' => 'required', 'Salary' => 'required', 'Description' => 'required',
            ]));
            return redirect()->route('offer.index');
        }
```

④ **Vue** (`resources/views/offer/index.blade.php`) : on affiche le bouton seulement si autorisé

```blade
@can('update', $offer)
    <a href="{{ route('offer.edit', $offer) }}">Modifier</a>
@endcan
```

⑤ **Test** : on vérifie qu'une autre entreprise est bloquée

```php
        public function test_company_cannot_edit_another_company_offer(): void
        {
            $a = User::factory()->create(['role' => 'company', 'company_id' =>
Company::create()->id]);
            $b = User::factory()->create(['role' => 'company', 'company_id' =>
Company::create()->id]);
            $offer = Offer::create(['company_id' => $a->company_id, 'Name' => 'Dev', 'Salary'
=> 1, 'Description' => 'x']);

            $this->actingAs($b)->get(route('offer.edit', $offer))->assertForbidden(); // 403
        }
```

Résumé du parcours :

```
Requête → auth (connecté ?) → IsCompany (entreprise ?) → Policy (mon offre ?) → page
            └ non : /login       └ non : 403                 └ non : 403
```

8. Questions probables à l'évaluation

Q : Différence entre authentification et autorisation ? R : L'authentification vérifie **qui** est l'utilisateur (login). L'autorisation vérifie **ce qu'il a le droit** de faire.

Q : Où est stocké le rôle dans JobPilot ? R : Dans la colonne `role` de la table `users` (`enum` : `candidat`, `company`, `admin`).

Q : Comment empêche-t-on de s'inscrire comme admin ? R : Dans `CreateNewUser`, avec la règle `Rule::in(['candidat', 'company'])`.

Q : Que fait le middleware `IsCompany` ? R : Avant le contrôleur, il vérifie que `role === 'company'`. Sinon, il renvoie une erreur 403 avec `abort(403)`.

Q : Où déclare-t-on les alias de middleware ? R : Dans `bootstrap/app.php`, avec `$middleware->alias([...])`.

Q : Différence entre Gate et Policy ? R : Une **Gate** est une règle isolée avec un nom (ex. : `access-admin`). Une **Policy** est une classe qui regroupe les règles **d'un modèle** (ex. : `OfferPolicy`).

Q : Où déclare-t-on une Gate ? R : Dans `AppServiceProvider::boot()`, avec `Gate::define('nom', fn (User $user) => ...)`.

Q : Comment Laravel relie `Offer` à `OfferPolicy` ? R : Automatiquement, grâce à la convention de nommage `Models\Offer` → `Policies\OfferPolicy`.

Q : À quoi sert `before()` dans une Policy ? R : Elle s'exécute avant les autres méthodes. On s'en sert souvent pour donner **tous les droits à l'admin** : elle renvoie `true` pour l'admin et `null` pour les autres.

Q : Pourquoi le middleware de rôle ne suffit-il pas ? R : Il vérifie le **rôle**, pas la **donnée**. Une entreprise pourrait modifier l'offre d'une autre. La Policy, elle, compare `$user->company_id` et `$offer->company_id`.

Q : `@can` dans une vue suffit-il à protéger une page ? R : **Non.** Il cache seulement le bouton. Il faut aussi protéger la route ou le contrôleur.

Q : Différence entre 403 et redirection vers /login ? R : Redirection vers /login : **pas connecté**. 403 : connecté, mais **pas le droit**.

Q : Que vaut `authorize()` dans les FormRequests de JobPilot ? R : `true`, donc tout le monde est autorisé. On peut y mettre `$this->user()->can(...)`.

Q : Les Policies de JobPilot sont-elles utilisées ? R : Non. Elles existent, mais toutes leurs méthodes renvoient `false` et ne sont jamais appelées. Ce sont les middlewares qui font le contrôle.

9. Mémo final (à relire juste avant)

```
RÔLE        = users.role  (admin | company | candidat)
MIDDLEWARE  = barrière sur la route       → vérifie le RÔLE      → 403
GATE        = règle avec un nom           → Gate::define(...)    → AppServiceProvider
POLICY      = règles d'un modèle          → OfferPolicy          → vérifie la DONNÉE
@can        = cache un bouton dans la vue → NE PROTÈGE PAS
```

Je veux…	J'écris…
Être connecté	`->middleware('auth')`
Réserver à un rôle	`->middleware('IsCompany')`
Créer une Gate	`Gate::define('access-admin', fn (User $u) => $u->role === 'admin');`
Créer une Policy	`php artisan make:policy OfferPolicy --model=Offer`
Vérifier dans le contrôleur	`Gate::authorize('update', $offer);`
Vérifier sur la route	`->can('update', 'offer')`
Vérifier en PHP	`$user->can('update', $offer)`
Afficher un bouton	`@can('update', $offer) ... @endcan`
Création (pas d'objet)	`Gate::authorize('create', Offer::class);`
Admin a tous les droits	`before()` → `return true;` si admin

Fichiers à connaître dans JobPilot :

Fichier	Contenu
`app/Http/Middleware/IsAdmin.php` (et `IsCompany`, `IsCandidat`)	les 3 middlewares de rôle
`bootstrap/app.php`	les alias des middlewares
`routes/web.php`	les routes et leurs middlewares
`app/Actions/Fortify/CreateNewUser.php`	le choix du rôle à l'inscription
`app/Policies/OfferPolicy.php`	la Policy des offres
`app/Providers/AppServiceProvider.php`	l'endroit où déclarer les Gates